

Hospital Management System Security and Privacy Case Study

Secure Software Engineering — a.a. 2025–2026

Leonardo Moro [869146] · Domenico Scalise [865702]

University of Bari



System Overview

The HMS is a PHP/MySQL web application centralizing hospital workflows across three user roles, exposing a broad dynamic input surface.

Key Facts

- Three user roles: Patient, Doctor, Administrator
- Backend: PHP + MySQL on XAMPP / localhost
- Entry points: func.php · func1–3.php · admin-panel.php · doctor-panel.php



Patient

- View and manage personal medical records
- Book and track appointments
- Access prescriptions & billing / PDF output



Doctor

- Manage schedules and availability
- Handle clinical requests and prescriptions
- Access doctor-panel.php workflows



Administrator

- Full system control and oversight
- User management across all roles
- admin-panel.php, admin-panel1.php



Technology Stack



Key Entry Points

- func.php · func1.php · func2.php · func3.php
- admin-panel.php · admin-panel1.php
- doctor-panel.php · prescribe.php
- Registration, login/auth, appointment booking, search endpoints, PDF generation

Why Security and Privacy Matter

The HMS handles sensitive PII and PHI-like data — any compromise threatens the full CIA triad and GDPR obligations.



Confidentiality

SQL Injection (CWE-89)

SQL Injection enables attackers to bypass authentication and extract PII/PHI-like records from the database — directly violating patient data confidentiality.

GDPR Violation

Trust Loss



Integrity

CSRF · IDOR (A01)

CSRF and IDOR-like vulnerabilities allow unauthorized alteration of appointments and medical records — enabling forged actions without user awareness.

Record Tampering

Forged Actions



Availability

Stored / Reflected XSS (A03)

XSS-driven session hijacking and abusive queries can disrupt hospital workflows — rendering the system unusable for medical staff and patients.

Session Hijack

Ops Disruption



Privacy is a **design property, not an afterthought** — a GDPR-oriented healthcare context mandates privacy-by-design from the earliest architectural decisions.

DATA AT RISK:

Patient Identities

Appointments

Prescriptions

Billing Records

Credentials

Methodology

Shift-Left, Secure-SDLC pipeline combining SAST, DAST, manual validation, and POSD-guided privacy re-engineering.

☰ 6-phase methodology: Passive Recon → Privacy Re-engineering

📈 SAST: 480 → 153 issues across 4 progressive scans

1 Passive Recon / Red Teaming

🔍 Repository & Config Inspection

- Repository inspection
- Tech fingerprinting
- Config-level observations



2 Active Recon / OWASP ZAP

🕷️ Site-Tree Crawling

- HTTP response analysis
- Form/parameter enumeration
- Attack surface mapping



3 SAST / Fortify SCA

</> Structural Vulnerability Analysis

- 4 progressive scans
- 480 → 153 issues
- 0 Critical (final scan)

4 DAST / OWASP ZAP

⚡ Runtime Behavior Simulation

- Black-box / gray-box
- Attack surface mapping
- Runtime exploit validation



5 Manual Validation / XAMPP

✍️ Before/After Regression Tests

- Real payload execution
- Behavioral verification
- Localhost regression tests



6 Privacy Re-engineering / POSD

🛡️ POSD + Privacy Knowledge Base

- Privacy strategy selection
- Pattern-driven architecture
- Privacy-by-design target

Threat Model and Attack Surface

The attack surface is dominated by dynamic, user-controlled inputs across authentication, search, administrative, and prescription workflows.

Reconnaissance



Passive Recon

PHP / MySQL / XAMPP fingerprinting · Exposed repo structure · Root DB user, empty password



Active Recon — OWASP ZAP

Site-tree crawling · HTTP response analysis · Form & parameter enumeration



Kill Chain: Reconnaissance precedes exploitation — surface mapping before attack

Threat Actors



External Attackers



Malicious Insiders



Cross-Role Privilege Abusers

Attack Surface — Entry Points

Authentication Handlers

func.php

func1.php

func3.php

Registration & Booking

func2.php

Search Endpoints

patientsearch.php

doctorsearch.php

appsearch.php

mesearch.php

Admin / Doctor Panels

admin-panel.php

admin-panel1.php

doctor-panel.php

Prescription & Billing

prescribe.php

PDF generation via TCPDF

Contact & Feedback

contact.php

Main Vulnerabilities Found

Seven vulnerability classes identified — spanning injection, session management, privacy exposure, and access-control weaknesses — all mapped to OWASP Top 10.

-  7 Vulnerability Classes Identified
-  All Mapped to OWASP Top 10

Vulnerability	Affected Files	CIA Impact	OWASP	Severity
SQL Injection CWE-89	func.php, func1–3.php, search handlers	Confidentiality Integrity	A03	Critical
Stored XSS CWE-79	admin-panel.php, doctor-panel.php rendering tables	Integrity Availability	A03	High
Reflected XSS CWE-79	Search endpoints, form echo paths	Integrity Availability	A03	High
CSRF CWE-352	State-changing forms in all panels	Integrity	A01	High
Plaintext Password Storage CWE-256	func2.php, func3.php, credential comparison logic	Confidentiality	A07	Critical
Privacy Overexposure CWE-359	Admin/search views rendering credential data	Confidentiality	A02	Medium
Weak Session & IDOR-like CWE-384 / CWE-639	session_start() patterns, GET-driven actions with ID param s	Confidentiality Integrity	A07/A01	High

Vulnerabilities were not merely theoretical — authentication bypass, data exfiltration, CSRF-driven tampering, and session abuse formed realistic attack chains.

 7 realistic attack scenarios documented

 Multi-step chain: SQLi → credential harvest → panel pivot (C+I+A)

Attack	Technique	CIA Impact
Authentication Bypass	SQL Injection tautology in login forms	C Unauthorized access to patient/doctor/admin sessions
Data Exfiltration	SQLi on search / contact endpoints	C PII and PHI-like record extraction
Time-Based Blind SQLi	Inference on partially hardened views	C Backend schema reconstruction
CSRF on Management Actions	Forged requests from malicious pages	I Unauthorized appointment booking, doctor record manipulation
Stored XSS → Session Hijacking	Injected script in rendered tables	A I Admin / doctor account takeover
IDOR-Like Manipulation	GET-parameter tampering on IDs	I Cross-user record access and unauthorized cancellations
Multi-Step Chain	SQLi → credential harvest → panel pivot	C+I+A All three CIA pillars compromised

Security Fixes Applied

Remediation layered across seven categories — from critical injection fixes to session hardening and privacy minimization

- SQL Injection
- Passwords
- </> XSS
- CSRF
- ↔ GET Actions
- 🔑 Sessions
- 🔒 Privacy

Category	Before (Vulnerable)	After (Remediated)
SQL Injection	Raw string concatenation in login, search & update flows	Prepared statements with parameterized queries across all flows
Password Handling	Plaintext storage and direct comparison in credential logic	bcrypt hashing with compatibility-aware legacy verification
XSS	Unencoded output rendering in admin/doctor tables and reflected paths	Output encoding (htmlspecialchars) on all major output surfaces
CSRF	No protection on any state-changing form submission	CSRF tokens generated and validated on all state-changing forms
GET Actions	Cancellations & updates via GET — no CSRF coverage	Migrated to POST with CSRF token validation enforced
Session Hardening	Scattered session_start() calls, no secure cookie flags	Centralized bootstrap logic with HttpOnly and Secure cookie flags
Privacy	Credential data visible in admin/search views — full exposure	Minimized output with role-based field visibility controls

Some empirical before/after regression tests on real payloads provide concrete runtime evidence that all major vulnerabilities — SQL Injection, session fixation, XSS, and IDOR — were effectively eliminated after remediation.

Test Case	Target / Area	Payload / Method	Before Remediation	After Remediation	Outcome
1 SQL Injection login bypass	Admin login (func.php)	' OR '1'='1	Login succeeded — auth bypassed by tautology	Login denied — admin panel not reached	Passed
2 Comment-based SQL Injection	Doctor login (func1.php)	ashok' -- / pwd: test	Login succeeded — WHERE clause truncated by comment	Login denied — doctor area blocked	Passed
3 Session fixation check	Session lifecycle (PHPSESSID)	Compare PHPSESSID before/after login via DevTools	Weak session lifecycle — ID not reliably regenerated	Session regenerated via sse_regenerate_session()	Passed
4 Post-logout access enforcement	Protected pages after logout	Login → copy URL → logout → direct URL reuse	Protected pages still reachable after logout	Redirected to public login/home page	Passed
5 Reflected XSS probe	Auth/input-driven flows	<script>alert('XSS')</script>	Structurally vulnerable — output echoed without encoding	Payload not executed — output encoding applied	Passed
6 IDOR / BOLA probe	Record identifiers (appts, patient IDs)	Manual ID manipulation in GET parameters	GET-driven IDs exposed IDOR-like risk surface	No successful exploitation via simple ID manipulation	Passed

Fortify Results Evolution

Across four scans, total issues fell from 480 to 153 through real code remediation and progressive alignment of scan scope to the deployable runtime.

- 1

Scan 1 — Baseline (729 files)

480 issues
- 2

Scan 2 — Intermediate

420 issues
- 3

Scan 3 — Penultimate (400 files)

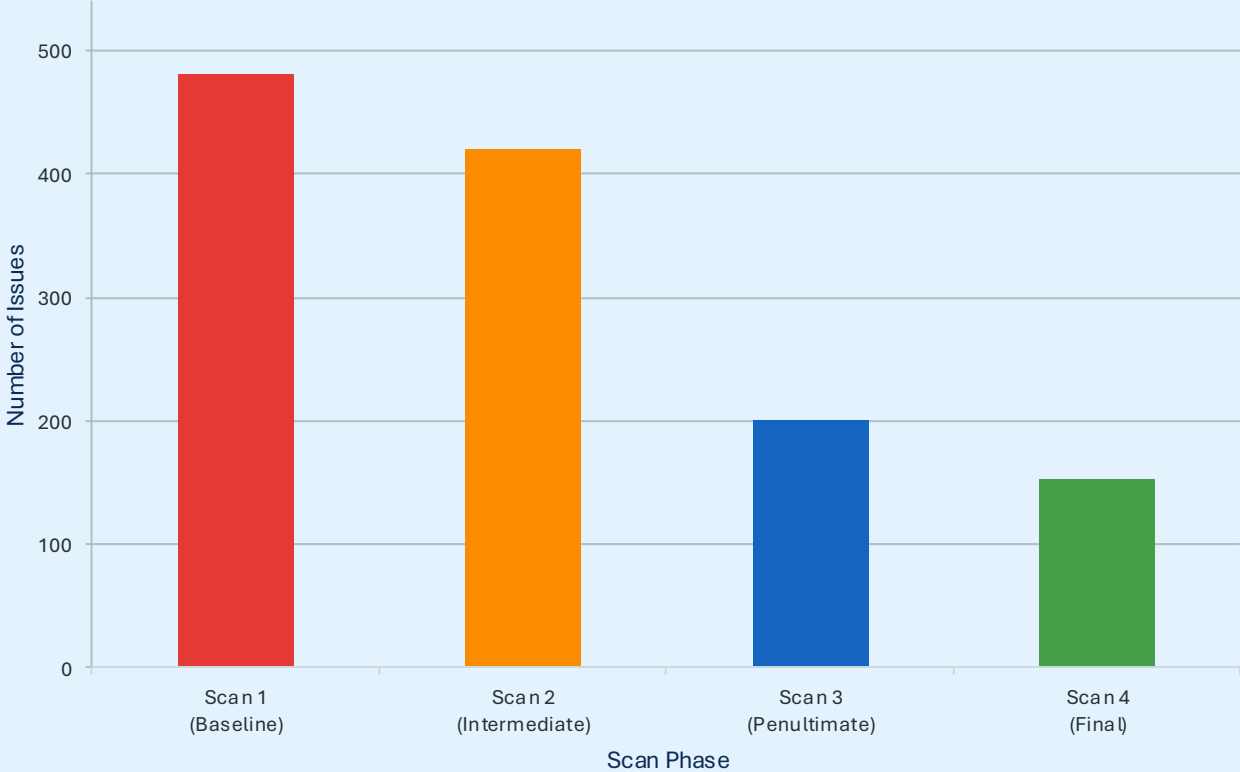
>153 issues
- 4

Scan 4 — Final · 0 Critical · 4 High

153 issues

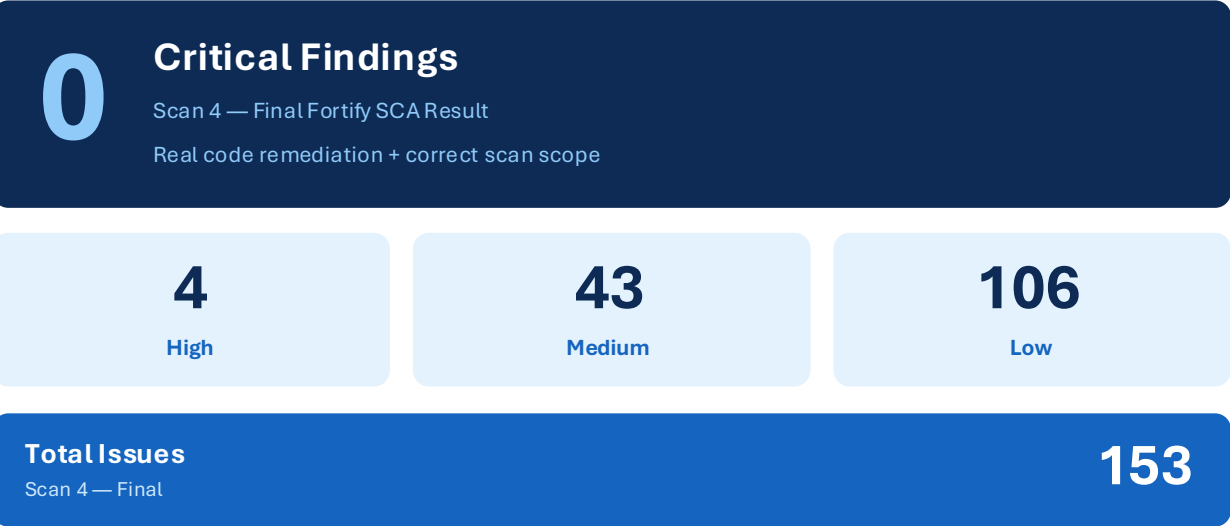
✓ 0 Critical = Code remediation + correct scan scope + removal of non-runtime artifacts

Total Issues per Fortify Scan

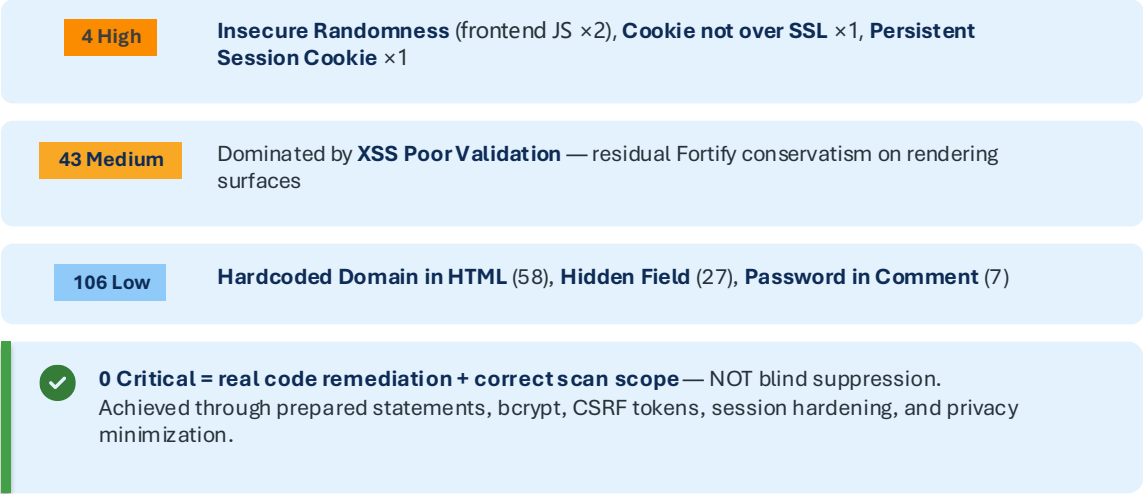


Final Fortify Snapshot

The final scan registers 0 Critical findings — a measurable outcome of secure coding improvements and precise scope definition.



Findings Breakdown — Scan 4



Using POSD and the Privacy Knowledge Base, seven privacy strategies and seven patterns define a role-based, privacy-by-design target architecture.

Privacy Strategies (PKB)

✖ Minimize

Hide

Separate

Inform

Control

Enforce

Demonstrate

Target Architecture

🛡️ GDPR / Policy Layer

🔒 Privacy-by-Design Controls

👤 Access Control Layer

📄 Application Logic (PHP)

🗄️ Data Layer (MySQL)

7 Privacy Strategies

7 Privacy Patterns

Privacy Patterns Applied		
Pattern	Purpose	Strategy
Selective Access Control	Enforce role-based data visibility	Hide / Enforce
User Data Confinement	Restrict data scope per role	Separate / Minimize
Personal Data Table	Centralize PII management with access control	Minimize / Control
Privacy Policy Display	Make data usage policies visible to users	Inform
Obtaining Explicit Consent	Collect affirmative consent at registration	Inform / Control
Informed Secure Passwords	Guide users toward strong credential practices	Enforce / Inform
Privacy Dashboard	Give users visibility and control over their data	Control / Demonstrate

Final Conclusions

The project transformed a severely vulnerable HMS into a measurably more secure, privacy-aware system — validated by code, runtime tests, and Fortify results.

Achievements

- ✓ All major attack vectors identified, validated, and mitigated
- ✓ SQL Injection eliminated via prepared statements — confirmed by manual before/after tests
- ✓ Password handling redesigned (bcrypt), CSRF tokens introduced, session hardened
- ✓ Privacy re-engineering grounded in POSD and Privacy Knowledge Base
- ✓ Secure SDLC applied end-to-end: Shift Left, SAST, DAST, manual validation, privacy architecture

Residual Risks

- **XSS surface** — residual Fortify conservatism on rendering paths warrants a final holistic review
- **Deeper authorization** — IDOR-like concerns on ID-driven endpoints require functional testing
- **Deployment hardening** — CSP headers, HTTPS enforcement, cookie Secure flag (deployment-level, not yet addressed)



💡 Security and privacy engineering are interconnected, iterative, and measurable — not one-time additions to the development lifecycle.